

Unit 1 · Topic Notes

Folder Structure Planning — MVC & Modularization

Course: Full Stack Application Development with MERN

Learning Objectives

1. Explain the MVC pattern and map it onto an Express backend
2. Set up a clean, scalable Node/Express folder structure
3. Set up a clean React frontend folder structure
4. Apply modularization principles instead of dumping everything in one file

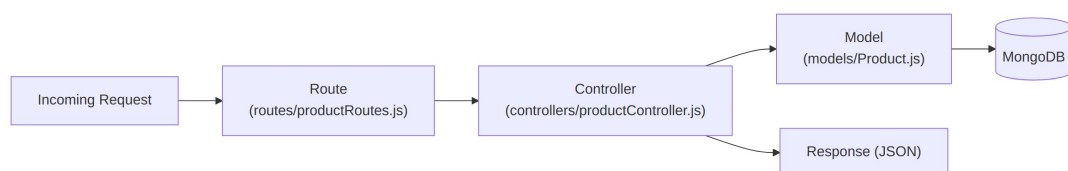
1. Why This Matters Before Writing Unit 2 Code

Every messy project you'll see in your career starts the same way — one `index.js` with 800 lines because nobody planned folders on day one. Fixing that starts here, before you write a single route.

2. MVC Pattern, Mapped to MERN

MVC Layer	Role	MERN Equivalent
Model	Data shape + database logic	Mongoose schemas (<code>models/Product.js</code>)
View	What the user sees	React components (separate app entirely in MERN)
Controller	Business logic connecting the two	Express controller functions (<code>controllers/productController.js</code>)

In a MERN app, “View” lives in the React app; the Express backend really implements **Model + Controller**, with **Routes** as a thin layer directing requests to controllers.



MVC request flow diagram

3. Standard Express/Node Folder Structure

```

server/
├─ config/
│   └─ db.js           # MongoDB connection setup
├─ controllers/
│   ├── productController.js
│   └─ userController.js
├─ models/
│   ├── Product.js
│   └─ User.js
├─ routes/
│   ├── productRoutes.js
│   └─ userRoutes.js
├─ middleware/
│   ├── authMiddleware.js
│   └─ errorMiddleware.js
├─ utils/
│   └─ generateToken.js
├─ .env
├─ .gitignore
├─ package.json
└─ server.js           # entry point — wires everything together

```

3.1 How the Pieces Connect — Code Walkthrough

`models/Product.js`

```

import mongoose from "mongoose";

const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  price: { type: Number, required: true },
  image: String,
});

export default mongoose.model("Product", productSchema);

```

`controllers/productController.js`

```

import Product from "../models/Product.js";

export const getProducts = async (req, res) => {
  const products = await Product.find();
  res.json(products);
};

```

`routes/productRoutes.js`

```

import express from "express";
import { getProducts } from "../controllers/productController.js";

const router = express.Router();
router.get("/", getProducts);

export default router;

```

`server.js`

```
import express from "express";
import productRoutes from "../routes/productRoutes.js";

const app = express();
app.use(express.json());
app.use("/api/products", productRoutes);

app.listen(5000, () => console.log("Server running on port 5000"));
```

Request flow: `server.js` → mounts `productRoutes` at `/api/products` → route calls `getProducts` controller → controller queries `Product` model → response sent back. This exact chain is what Unit 2/3 will build on top of.

4. Standard React Folder Structure

```
client/
├── src/
│   ├── components/          # reusable, dumb UI pieces
│   │   ├── Navbar.jsx
│   │   └── ProductCard.jsx
│   ├── pages/               # route-level views
│   │   ├── Home.jsx
│   │   └── ProductDetails.jsx
│   ├── hooks/               # custom React hooks
│   │   └── useFetch.js
│   ├── services/            # API call functions (Axios wrappers)
│   │   └── productService.js
│   ├── context/             # global state (Context API)
│   │   └── CartContext.jsx
│   ├── App.jsx
│   └── main.jsx
├── .env
└── package.json
```

Rule of thumb: `components/` = reusable and presentation-only, `pages/` = tied to a route and composes components, `services/` = the *only* place that talks to the backend (never call `fetch` / `axios` directly inside a component).

5. Modularization Principles

Principle	What it means	Anti-pattern
Single Responsibility	One file does one job	A <code>productController.js</code> that also sends emails and processes payments
Feature-based grouping (at scale)	Group by feature (<code>features/cart/</code>) instead of type, once the app grows large	Mixing this with type-based structure inconsistently
No logic in routes	Routes only map path → controller function	<code>router.get("/", async (req,res) => { /* 40 lines of logic */ })</code>
Config isolated	Secrets/env config in one place, never hardcoded	<code>mongodb://localhost/mydb</code> hardcoded inside a controller

For this course: **type-based structure** (as shown in §3–4) is the standard — it's simpler to grade and consistent across all student repos. Feature-based structure is what you'll see in larger production codebases — good to know about, not required here.

6. Hands-On Lab (30 min, individual)

1. Create the folder skeleton above for your own capstone's `server/` and `client/` directories (empty files are fine at this stage)

2. Write one dummy model, one controller, one route file, and wire them into `server.js` exactly like the walkthrough in §3.1 — but for your own capstone’s main entity (e.g., `Task`, `Item`, `Booking`)
 3. Push this skeleton to GitHub as your first `feature/project-setup` branch and open a PR (tying back to the Git/GitHub topic)
-

7. Quick Assessment

1. In MVC, which part of a MERN app is really the “View”?
 2. What should a route file *never* contain?
 3. Where should Axios/fetch calls live in the React structure, and why keep them out of components?
 4. What’s the difference between type-based and feature-based folder structuring?
 5. Trace the request flow: a `GET /api/products` request comes in — list the files it touches, in order.
-

Notes prepared for Full Stack Learning — MERN course, Unit 1.